SPINE++

An Iterative Approach for semantic and instance segmentation

Junjie Xia
SLAC
06/17/2026, NPML@UCI

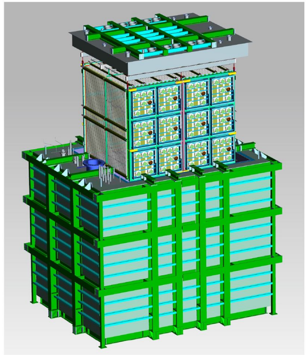
SLAC NATIONAL ACCELERATOR LABORATORY

Stanford University | U.S. DEPARTMENT OF ENERGY

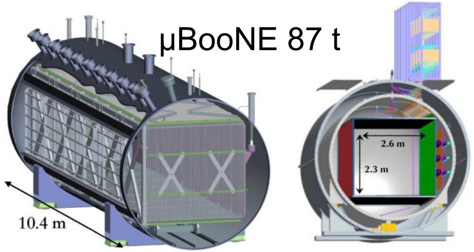
SPINE

Liquid Argon Time Projection Chamber (LArTPC)

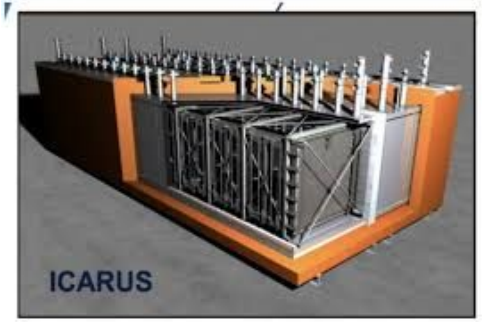
- A large volume of LAr with a uniform electric field between cathode and anode
- When charged particles traverse thru the LAr, they excite the Ar atoms forming Ar ions and ionized electrons:
    - The ionized electrons will be captured by the uniform electric field and drift toward the detector charge collection plane → **charge readout signals**
    - The de-excitation of Ar ions release scintillation photons that travel toward the optical sensors → optical readout signals
- The charge signals provide the **trajectory** and **calorimetric** information of the traversing particle.



SBND 112 t



µBooNE 87 t

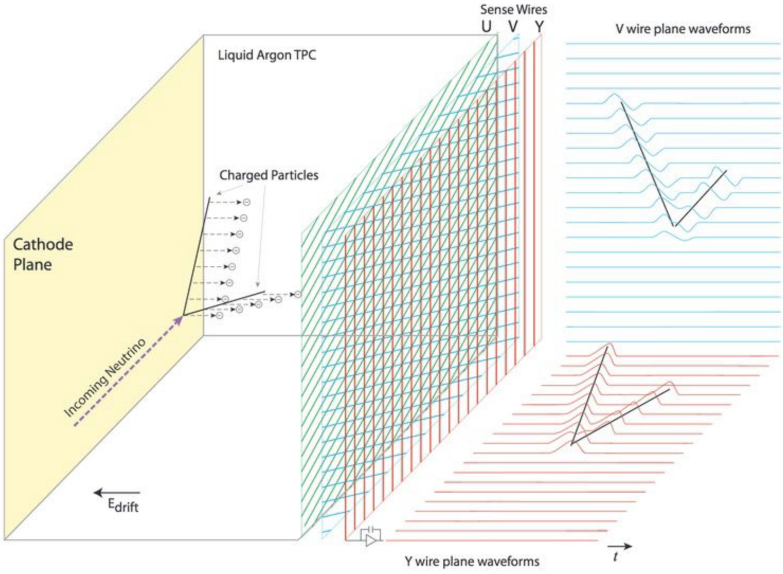


ICARUS 476 t

And DUNE ND-LAr ~150 t, FD ~40 kt

Charge signals LArTPC

arxiv.org/abs/1704.02927



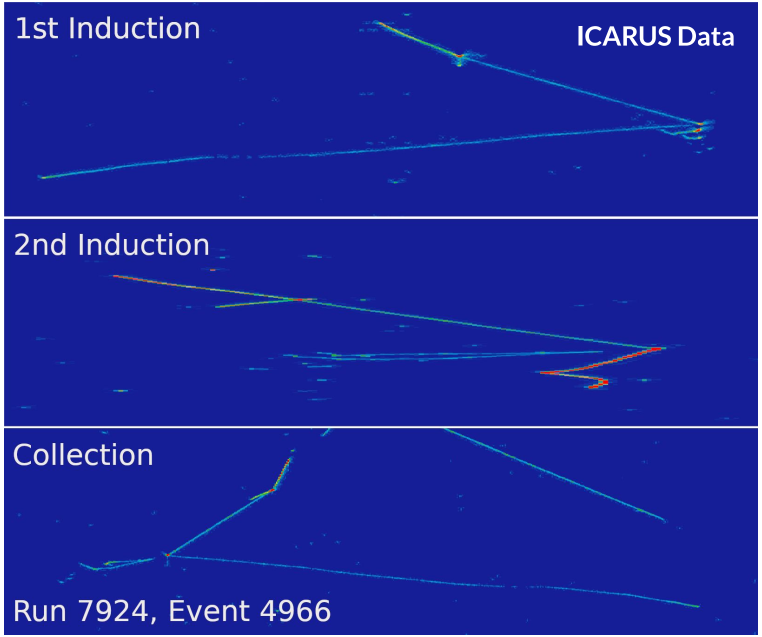
- Two types of charge collection plane:
  - **Wire** (SBND, μBooNE, ICARUS, DUNE FD)
  - Pixel (DUNE ND-LAr, 2x2)

- 3 wire planes oriented at different angles register the drifting electrons at anode.
  - First two planes have negative voltage applied and register the induction charge signals
  - The third plane collects electrons
  - Each pair of 2 planes forms a projection view of the 3D particle trajectories

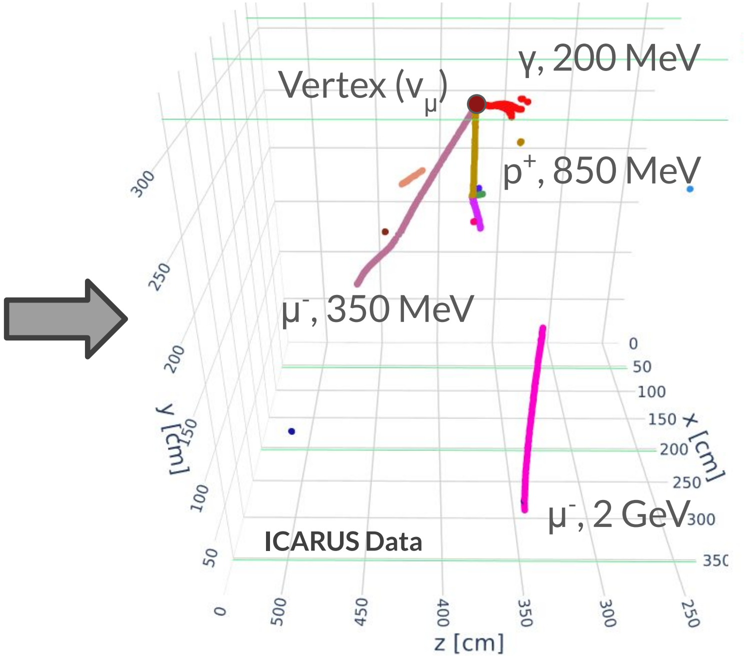
- O(1) mm spatial resolution depending on the wire spacing

Reconstruction goals

What we start with

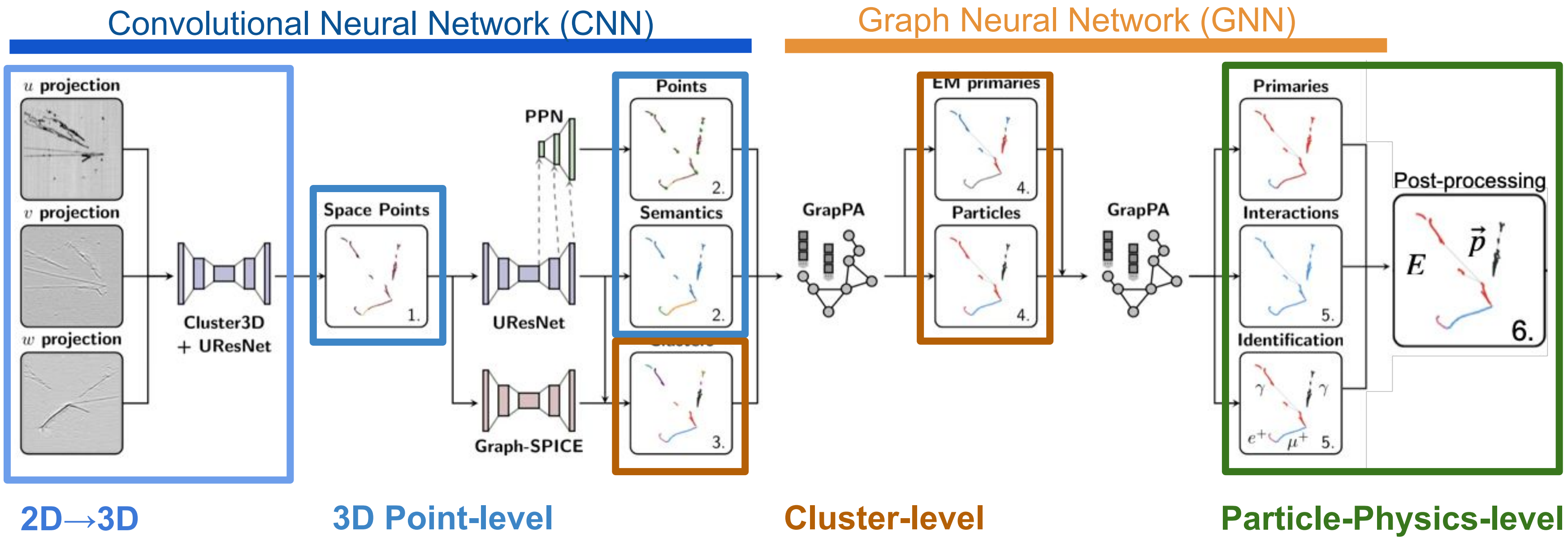


What we want (“particle flow”)



F.Drielsma’s talk at SPINE workshop 2025

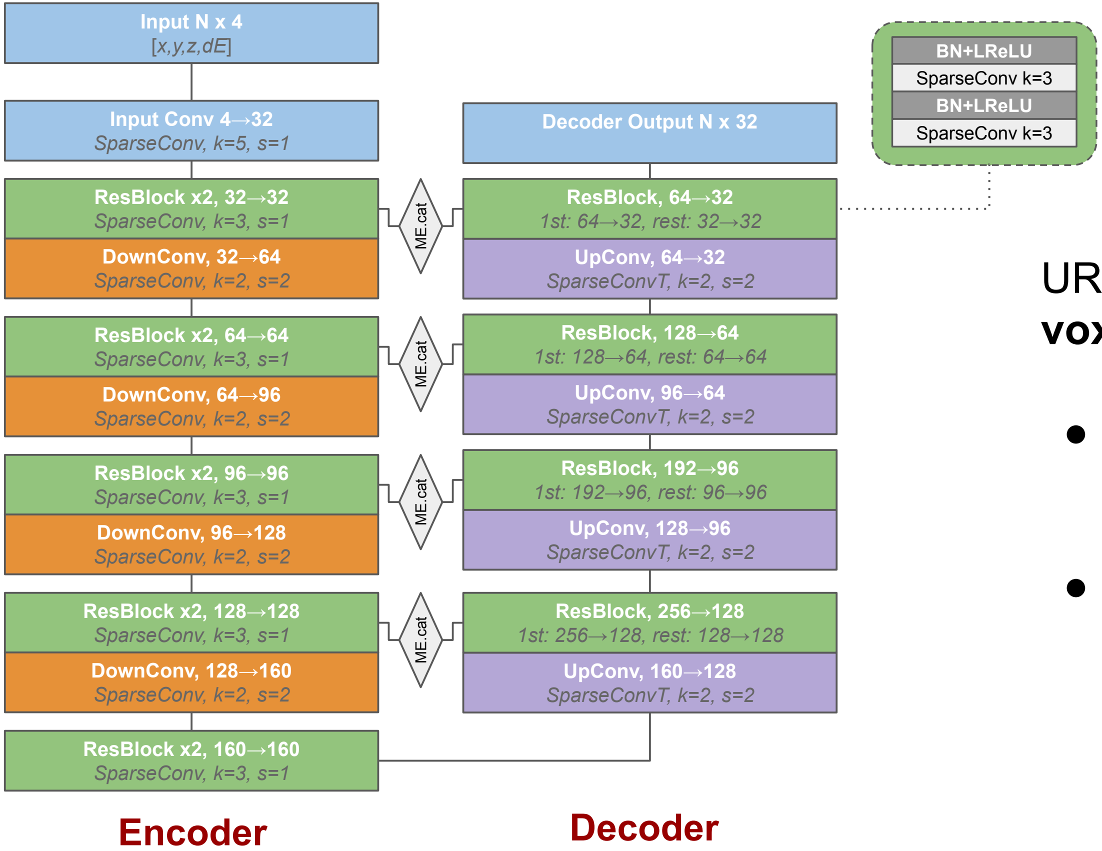
Scalable Particle Imaging with Neural Embeddings (SPINE)



SPINE

Paper: https://arxiv.org/abs/2102.01033
Github: https://github.com/DeepLearnPhysics/spine

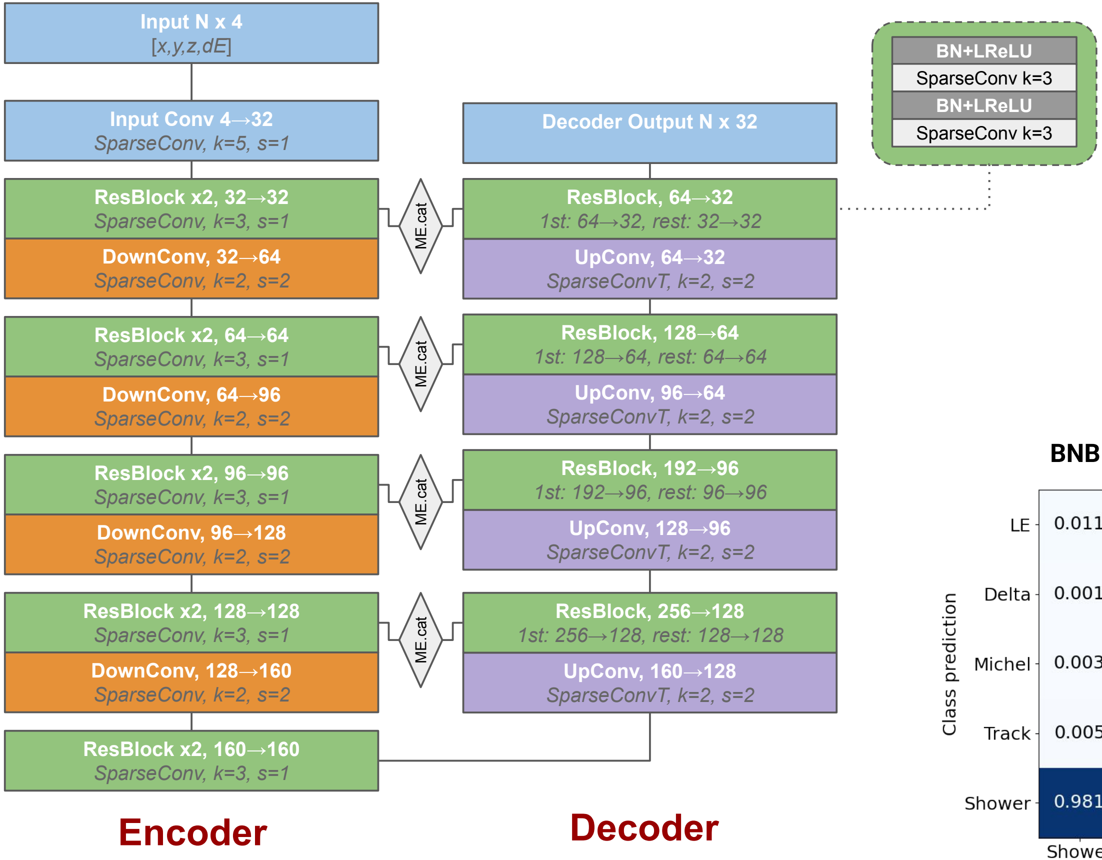
Current semantic segmentation performance of SPINE



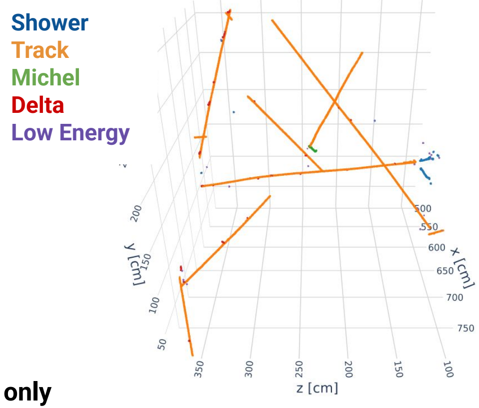
UResNet with Sparse Convolution on **voxelized volume** (3mm voxel edge)

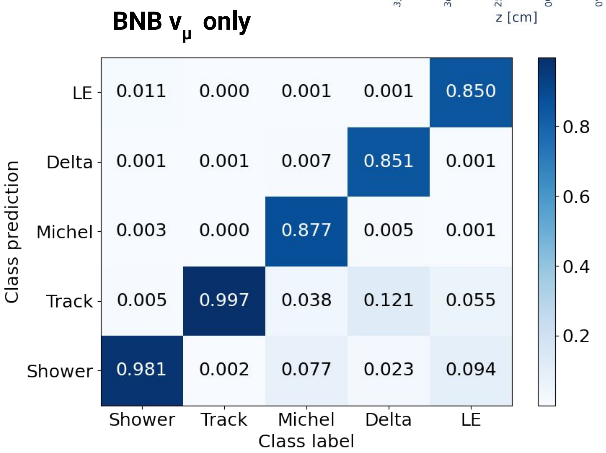
- 4D input: 3D voxel coordinates & energy deposition per voxel
- Default is 5 layer, 32-dim decoded feature space on full sparse resolution

Current semantic segmentation performance of SPINE



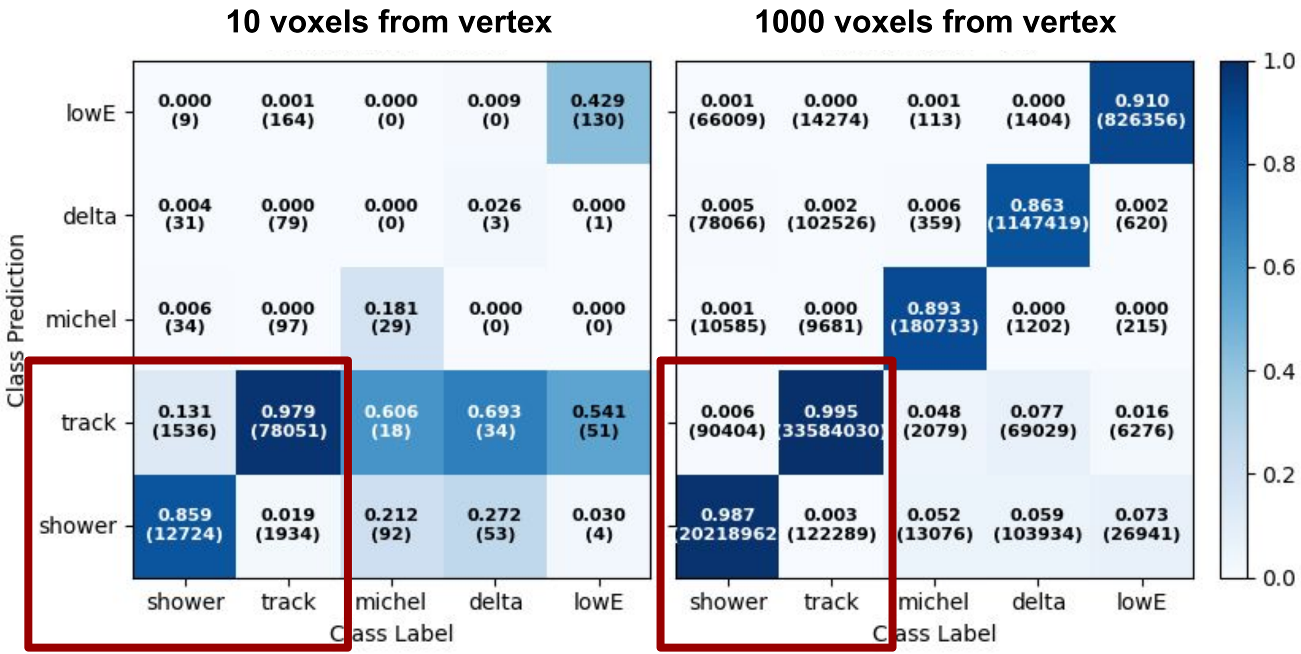
F.Drielsma's talk at SPINE workshop 2025





Paper: PhysRevD.102.012005

But if one takes a closer look…

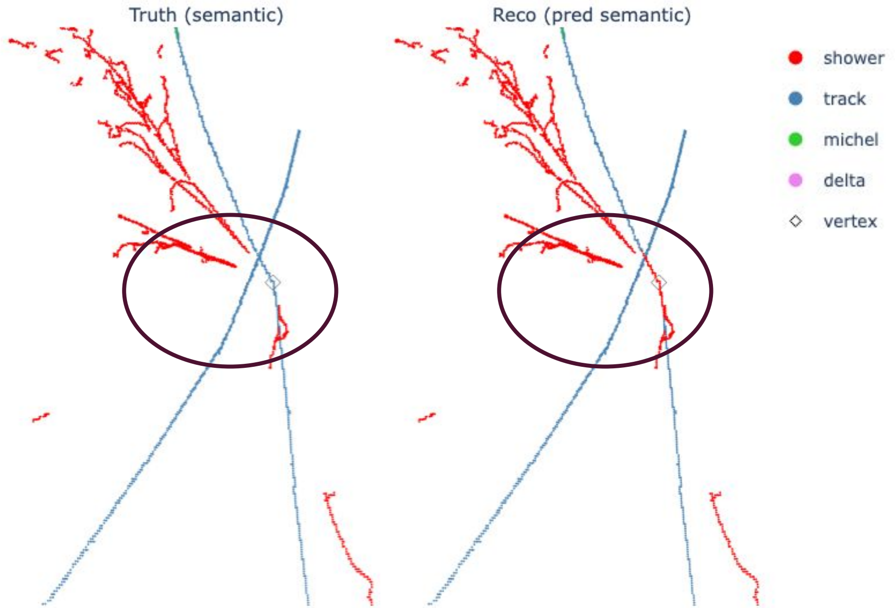


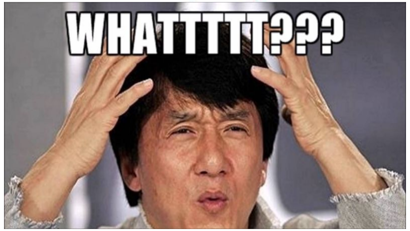
$$\text{Recall} = \frac{\text{TP}}{\text{TP+FN}}$$

Much deteriorated semantic segmentation performance in the vicinity of interaction vertices.

**Based on a different MC, NOT the one in PhysRevD.102.012005*

Around the vertex





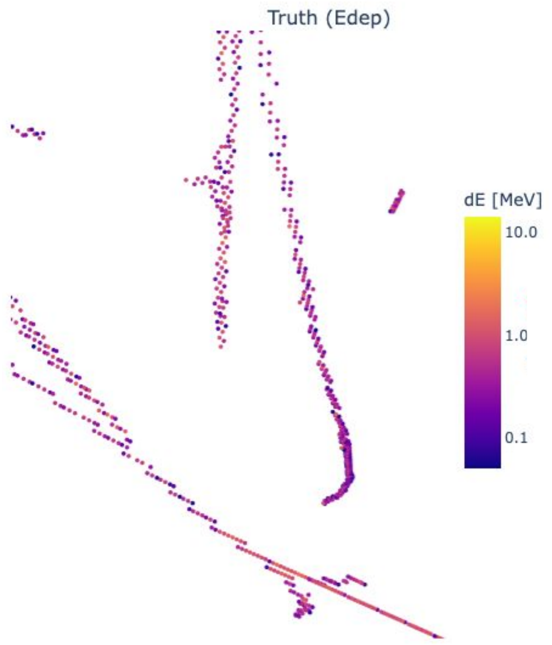
- The trajectory density around vertex is much higher than the whole detector
- Multiple trajectory types exist here, mostly shower and tracks
- The beginning of a electromagnetic shower (shower trunk) is visually similar to a track
- Given that segmentation is an early stage in the SPINE chain, this error will impact ALL the downstream clustering processes
    - **Electron/photon PID** is based on the beginning of a shower
    - Missing the particle start causes error in **energy reconstruction**

The challenge of instance segmentation

Methods that didn't work:

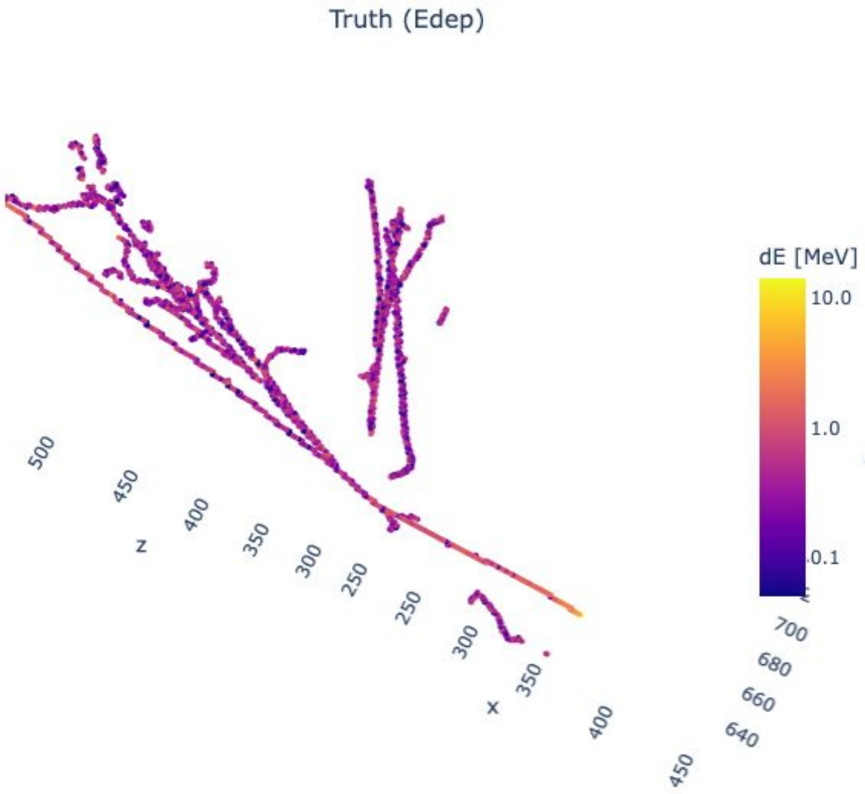
- Upweighting the loss on near-vertex voxels
- Deeper UResNet for larger receptive field

This is not an optimization problem, the model has reached its limit with the given features to learn the difference.



How many showers and tracks in this view?

The challenge of instance segmentation

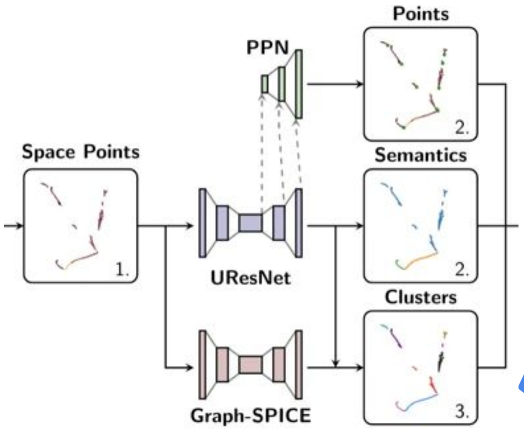


In fact even for experts' eyes, it only becomes easier if we look at the whole image instead of zooming in the near vertex region.

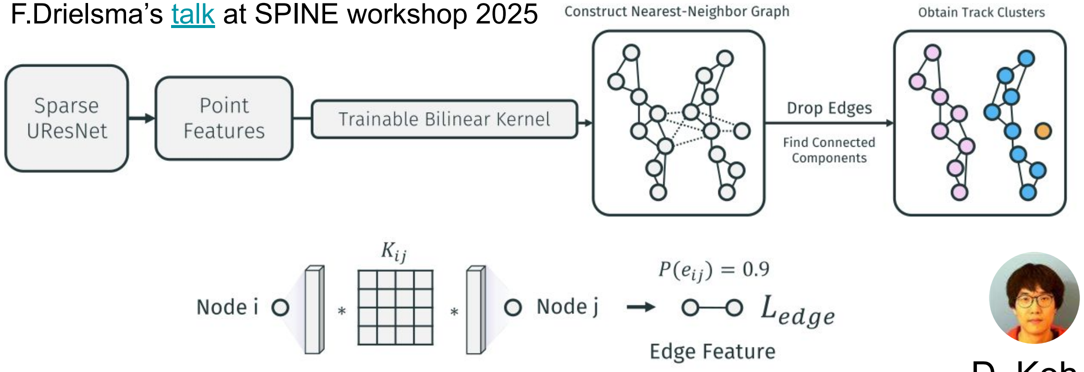
The shower and track become obvious as the particles propagate and we can see them as a whole, i.e. the feature needed for the near-vertex semantic segmentation is **not at voxel level, but instance level**

But how do we pass this instance-level information to the model?

Graph-SPICE in SPINE



F.Drielsma's talk at SPINE workshop 2025

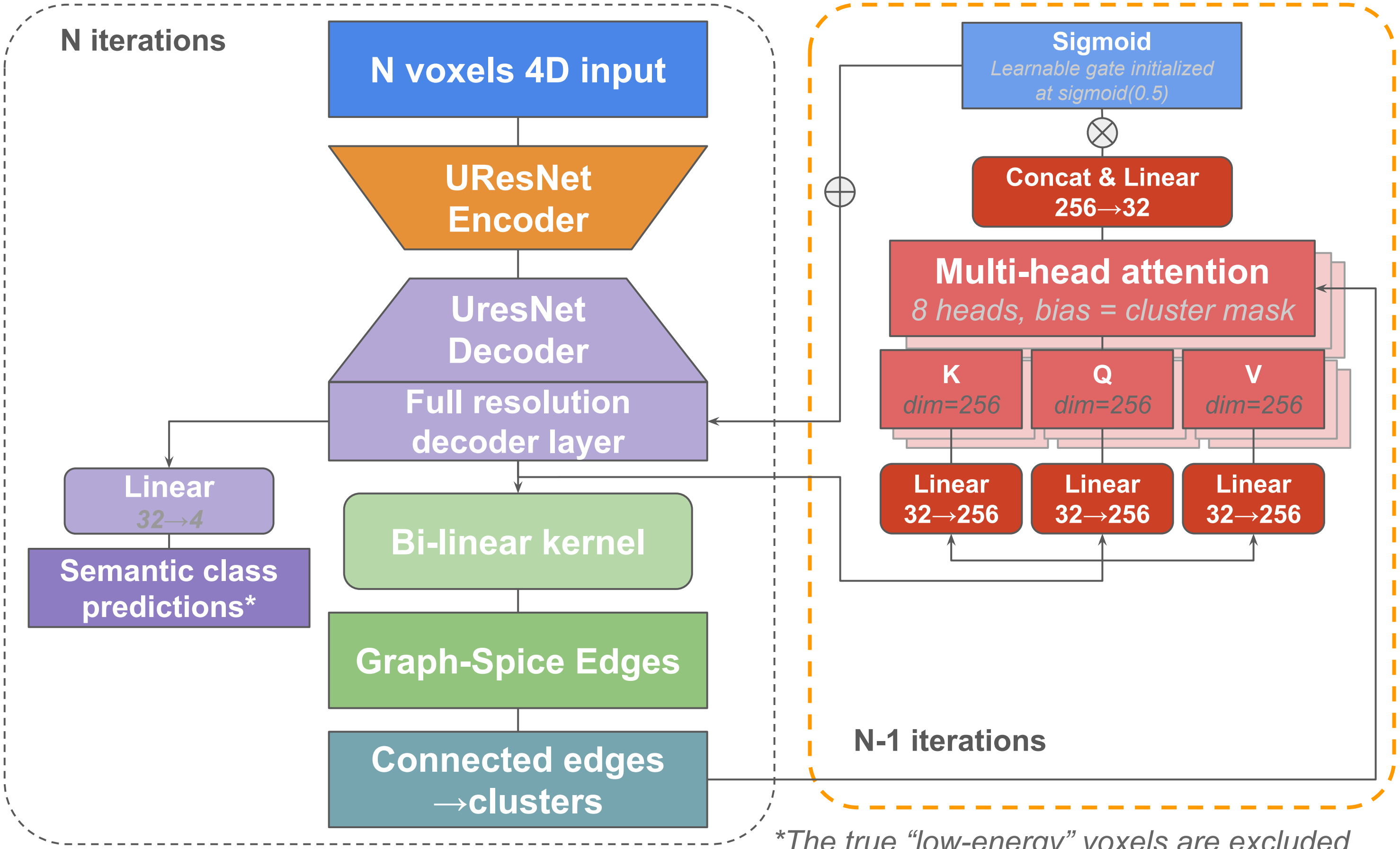


D. Koh

- From the UResNet decoder features, predict edge connections $P(e_{ij})$ between the ith and the neighbouring j=1,2,…,5 nodes via a trainable bilinear kernel
- Only the nodes of the same true semantic type can be connected by edges during the training
- Connection threshold $P(e_{ij}) > 0.9$

KNN is used in this work, but ball graph is also available and probably a better option

The Iterative Graph-SPICE



*The true "low-energy" voxels are excluded

*The default Graph-SPICE doesn't turn on the semantic prediction head

The Iterative Graph-SPICE

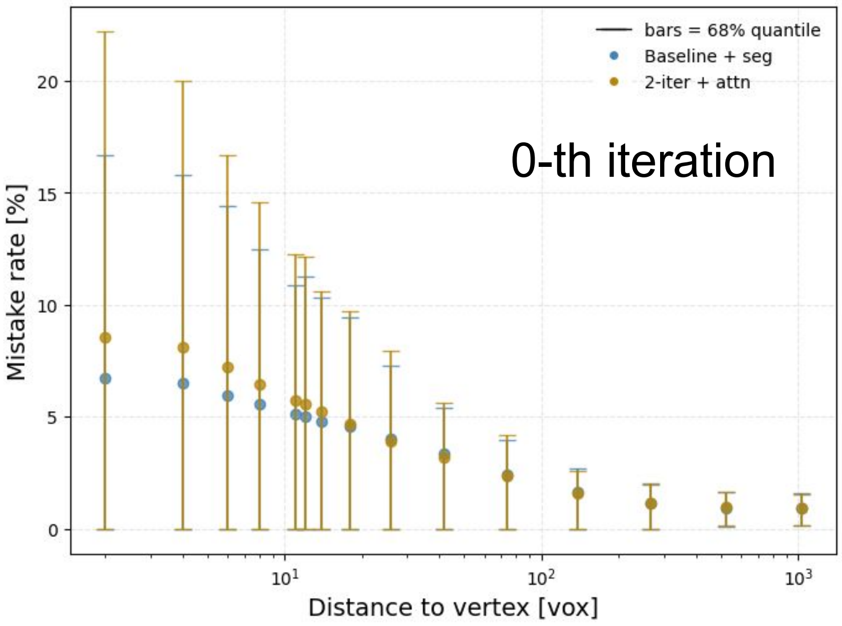
$$\mathcal{L} = \boxed{\mathcal{L}_{\text{edge}}} + \boxed{C_{\text{seg}}^{(0)}\mathcal{L}_{\text{seg}}^{(0)} + C_{\text{seg}}^{(\text{last})}\mathcal{L}_{\text{seg}}^{(\text{last})}}$$

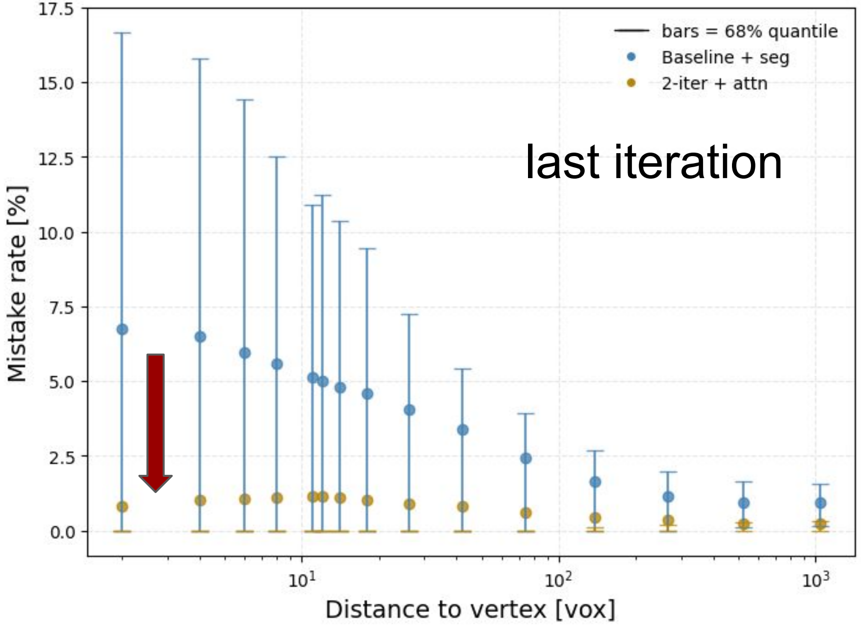
$$\boxed{\mathcal{L}_{\text{edge}} = 0.8\mathcal{L}_{\text{log-Dice}}(\hat{p}_e, t_e) + 0.2\mathcal{L}_{\text{BCE}}(\hat{p}_e, t_e)}$$

$$\boxed{\mathcal{L}_{\text{seg}}^{(k)} = \mathcal{L}_{\text{CE}}(\hat{y}_{\text{seg}}^{(k)} | y_{\text{truth}}),\ C_{\text{seg}}^{(0)} = 0.3,\ C_{\text{seg}}^{(\text{last})} = 0.5}$$

- The edge loss is only from the last iteration.
- The segmentation loss at 0-th iteration is on the semantic prediction without attention scores
- The attention scores affect the n>1 iteration segmentation, and only the last iteration segmentation is supervised.

Semantic segmentation performance



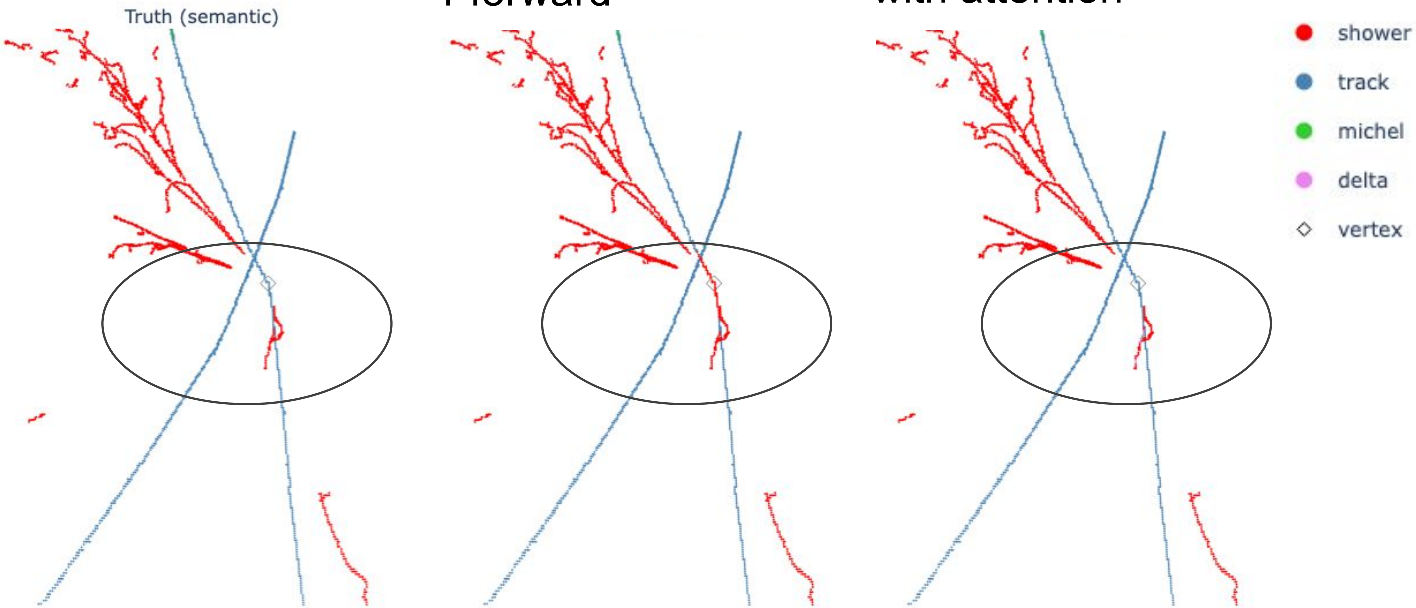


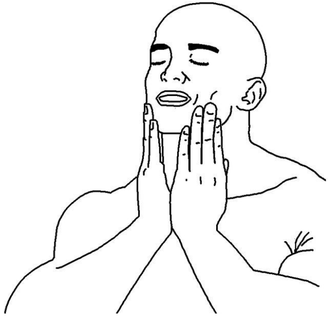
In the 2-forward (1 attention feedback loop) model, achieved up to **88% improvement** in segmentation mistake within a few voxels around the vertex, and **99% accuracy** for all checked voxel distances.

Semantic segmentation performance

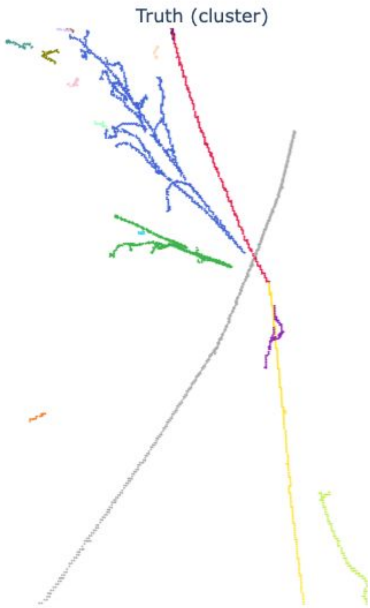
1 forward

2 forwards
with attention

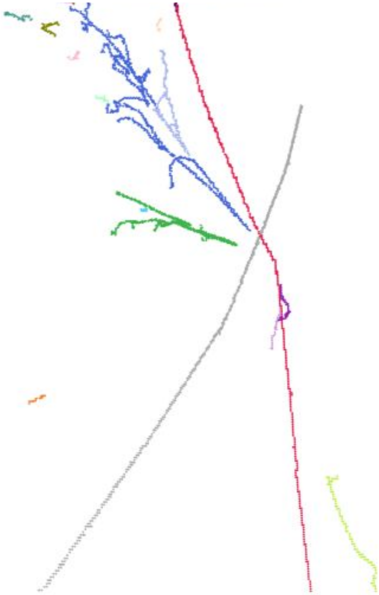




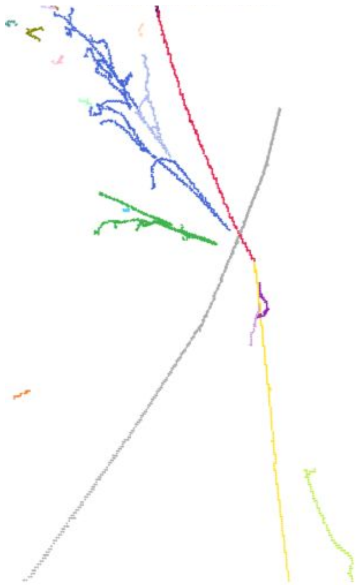
Instance segmentation performance

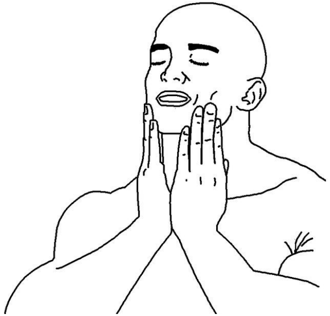


1 forward

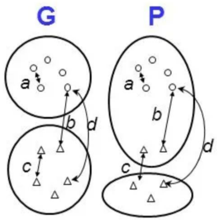


2 forwards
with attention





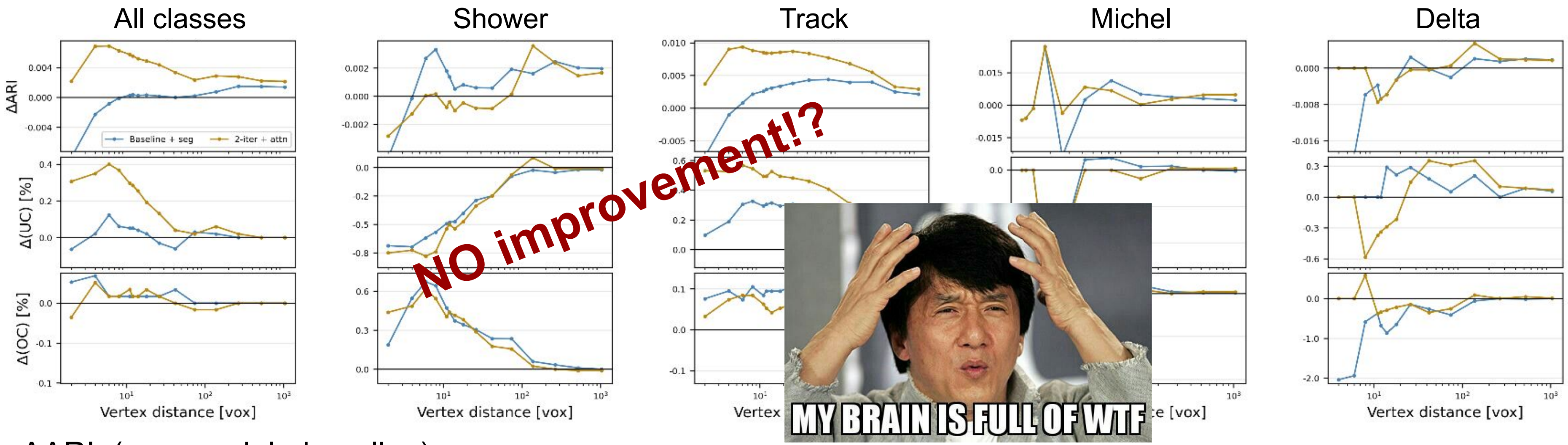
Instance segmentation performance



$$RI(P,G) = \frac{a+d}{a+b+c+d}$$

$$\boxed{ARI = \frac{RI - E(RI)}{1 - E(RI)}}$$

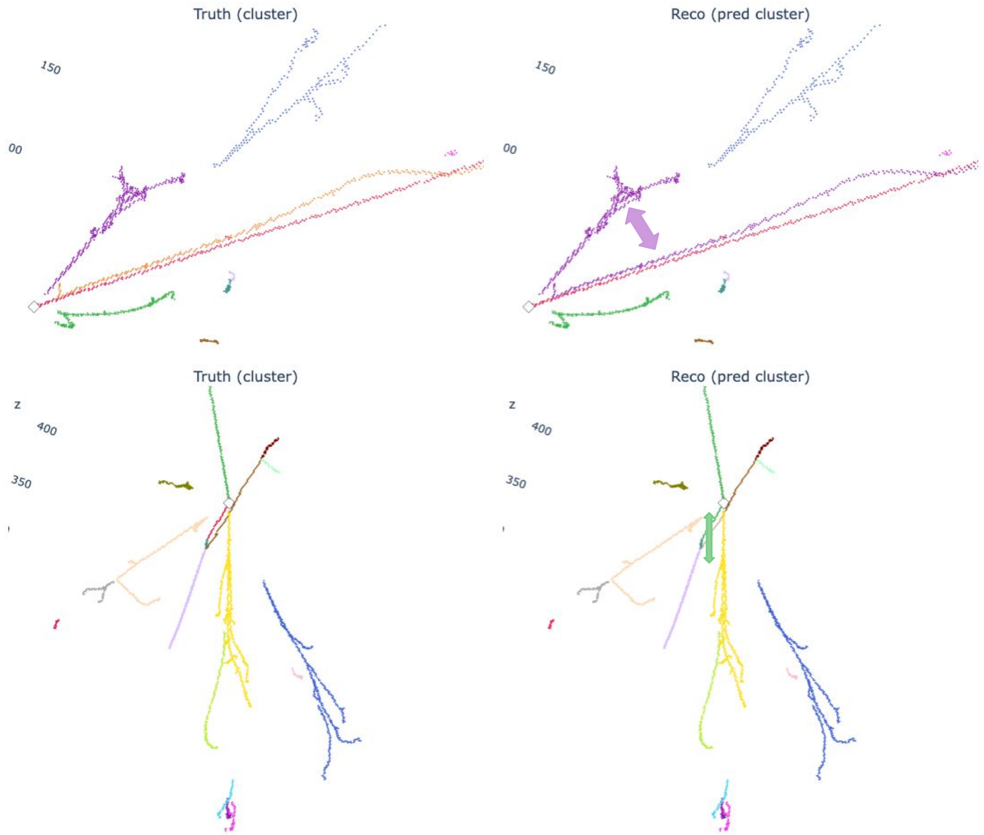
OC: (overclustering (n_pred > n_true)) & (n_true=1)
UC: (underclustering (n_pred < n_true)) & (n_pred=1)



ΔARI: (new model - baseline)
ΔOC/UC: (baseline - new model)

Clustering only done on the voxels of the same **truth semantic type*

Clustering failure modes



Common failure modes in both track and showers.

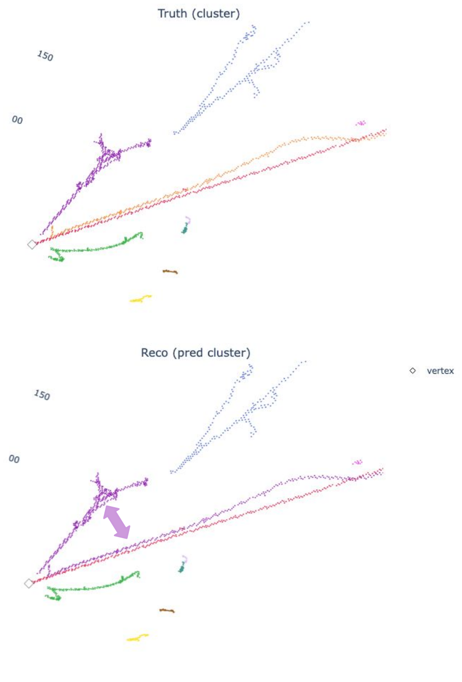
Underclustering in regions of high fragment density:

- Two showers ~touching around the start
- A scattered track, or short track but connected to a long one

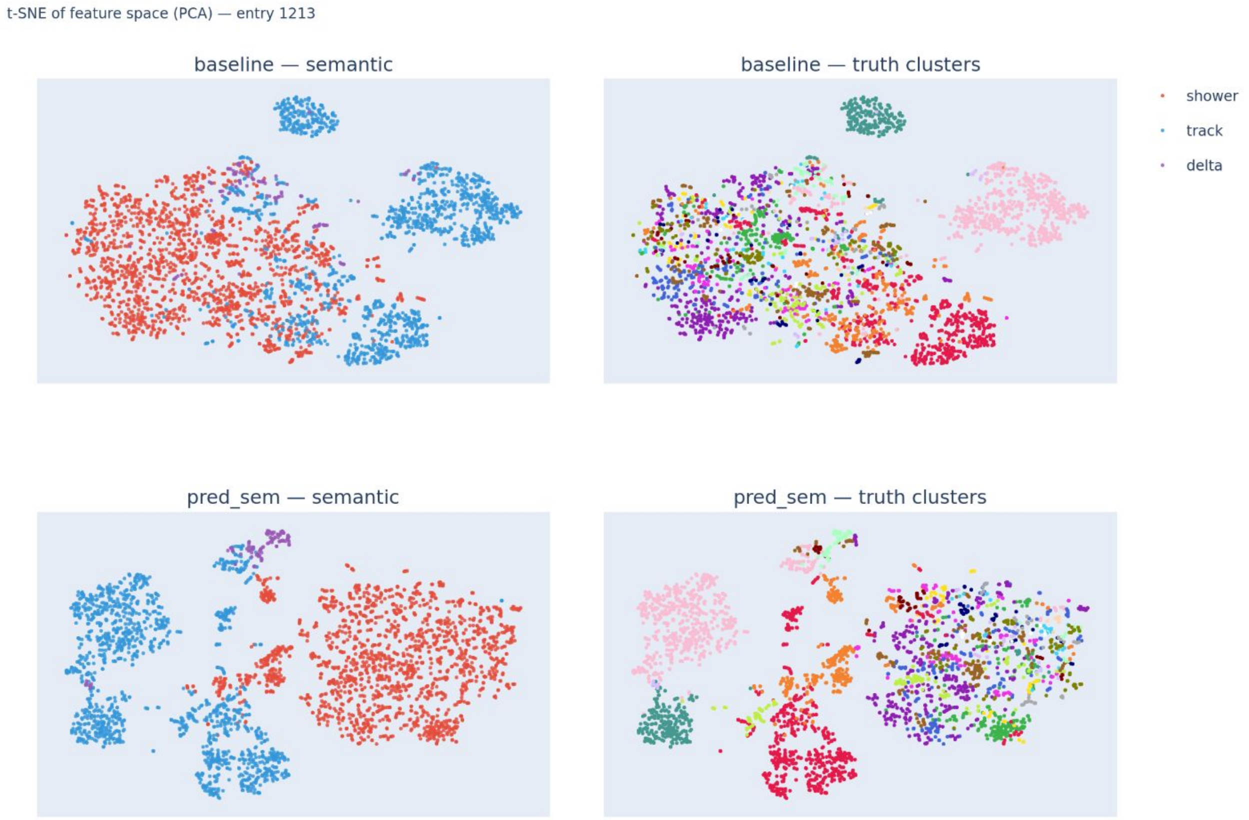
Overclustering also happens, but fixable downstream

These events were clustered perfectly in the 1 forward model.

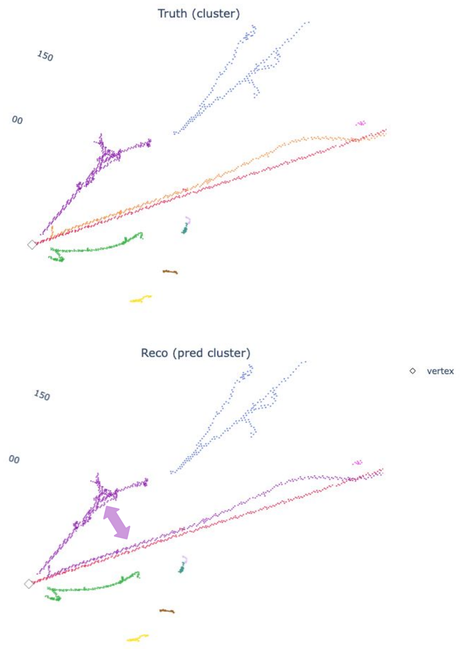
Looking at the feature space

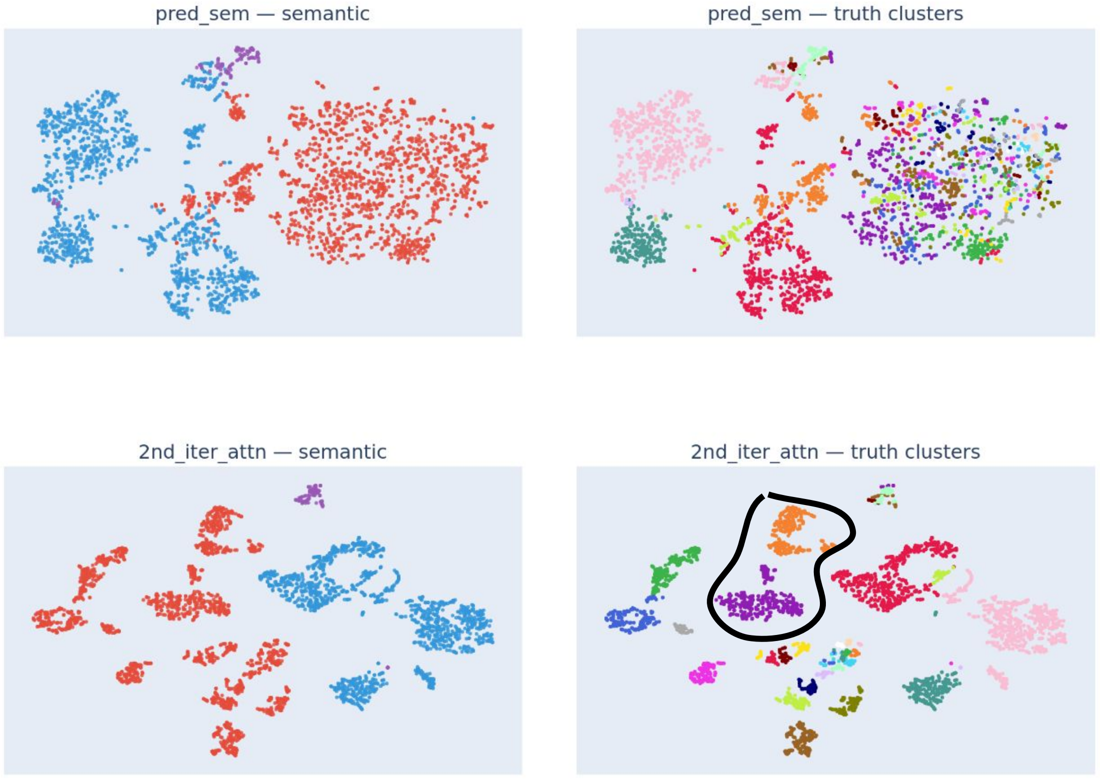


Semantic segmentation loss enhances the shower/track separation in the feature space →



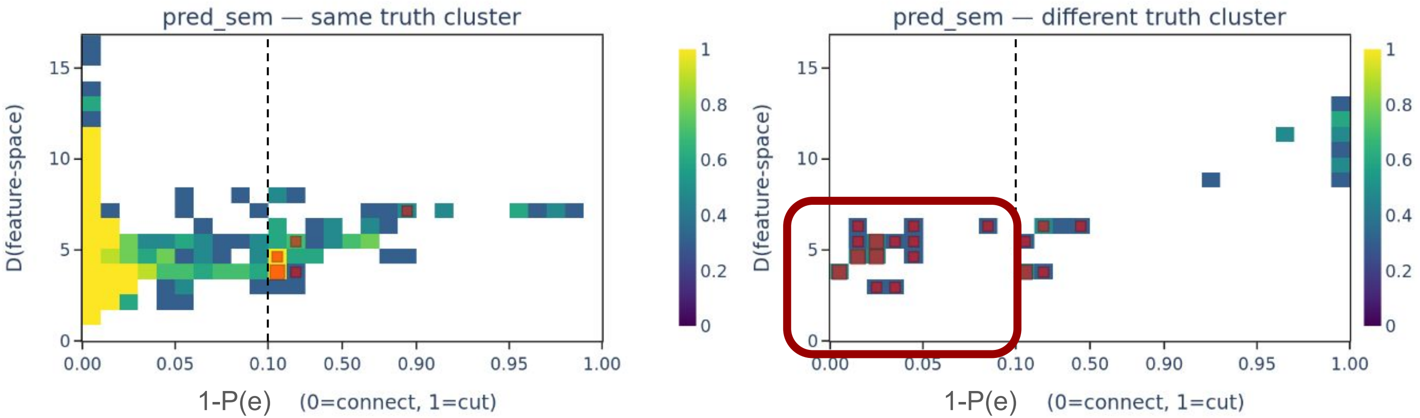
Looking at the feature space





Attention enhances the separation of different instances much more →

Feature space vs. edge probability



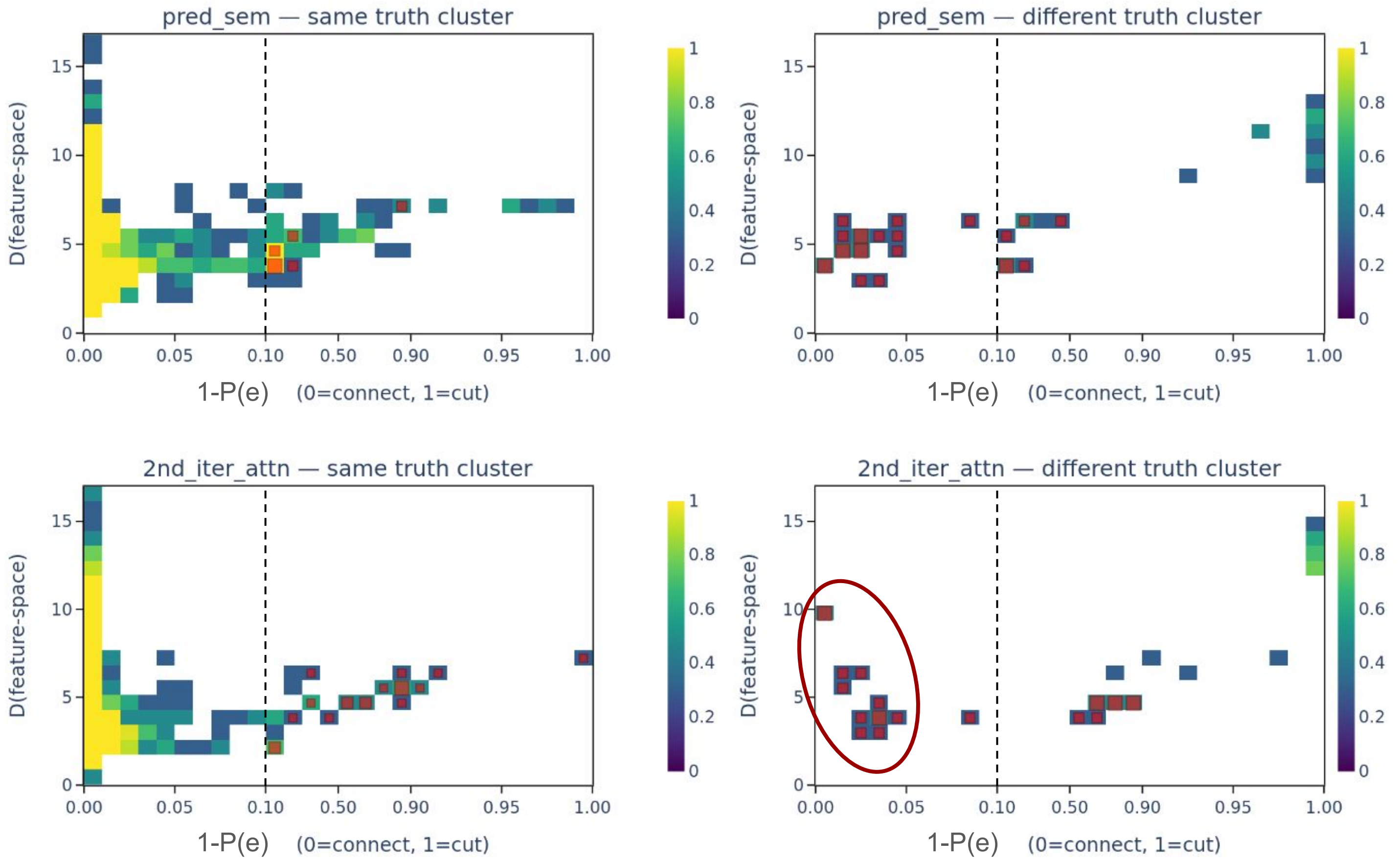
Tricky region: similar features and spatially close voxels with high edge probability, but should be different clusters.

Red squares: **prediction != truth** in each case

Edge_prob > 0.1 is the threshold that the connection between two nodes will never be considered. But the two can still be grouped as the same instance via a longer connection path.

Positive correlation between D(feature-space) & edge_prob indicates the decoder feature space and KNN are working in the same direction, otherwise the KNN-mapped edges disconnect from the feature space separation.

Feature space vs. edge probability



Even though the features are pushed apart, the edges are still connected, e.g. only need to mistake 1 edge connection on the boundary to merge two big clusters.

Trade-off of semantic and instance segmentation

Nested Analysis of Variance (ANOVA): $\mathrm{Var}_{\text{total}} = \underbrace{\mathrm{Var}_{\text{between-type}}}_{\text{semantic}} + \underbrace{\mathrm{Var}_{\text{between-inst, within-type}}}_{\text{instance}} + \underbrace{\mathrm{Var}_{\text{within-inst}}}_{\text{noise}}$

If the ratio $r = \frac{\mathrm{Var}_{\text{semantic}}}{\mathrm{Var}_{\text{instance}}}$

increases, the decoder feature space has rotated to semantic encoding and thus become dominated by semantic segmentation task.

In addition, check the principal angle cosine similarity of the semantic and instance LDA subspaces. If the projection vectors become more aligned, the two tasks are harder to distinguish from the feature space, e.g. instance encoding space more overlapped with the semantic encoding

Models	$\mathrm{Var}_{\text{semantic}}/\mathrm{Var}_{\text{total}}$	$\mathrm{Var}_{\text{instance}}/\mathrm{Var}_{\text{total}}$	Ratio	Cosine similarity
Baseline	16.5%	7.5%	2.2	0.50
Baseline + seg 1 forward	**48.8%**	**4.3%**	**11.4**	0.57
Baseline + attn 2 forward	**55.0%**	**5.5%**	10.0	0.55

Trade-off of semantic and instance segmentation

Nested Analysis of Variance (ANOVA):

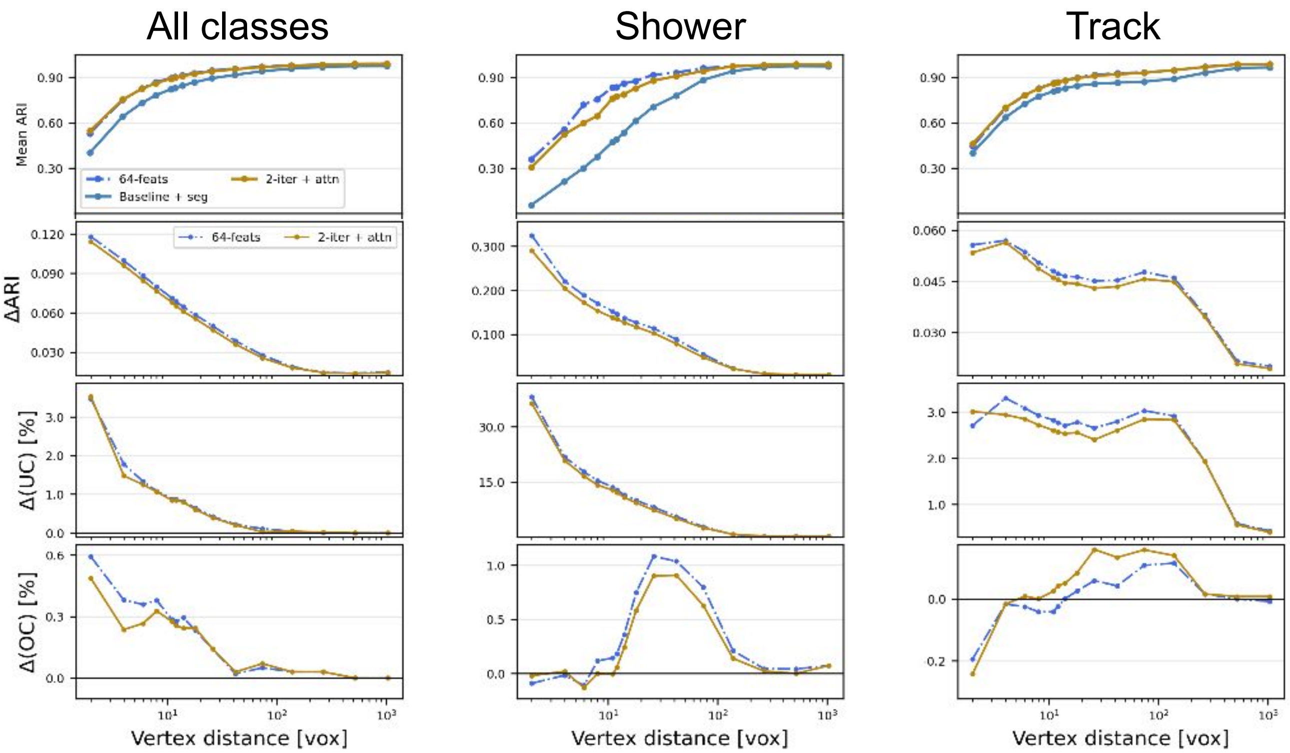
$$\mathrm{Var}_{\text{total}} = \underbrace{\mathrm{Var}_{\text{between-type}}}_{\text{semantic}} + \underbrace{\mathrm{Var}_{\text{between-inst, within-type}}}_{\text{instance}} + \underbrace{\mathrm{Var}_{\text{within-inst}}}_{\text{noise}}$$

- The direct supervision on segmentation task heavily shift the decoder space. The main variance is by far in the different semantic types.
  - 4 semantic class throughout all events
  - O(10~100) clusters varied event-by-event
- Attention helps enhance both variances
- Expanding the feature space allows more freedom to separate the two tasks and resolve the trade-off.
- Yet a large feature space will also have too much freedom that ends up in $\mathrm{Var}_{\text{within-inst}}$

Models	$\mathrm{Var}_{\text{semantic}}/\mathrm{Var}_{\text{total}}$	$\mathrm{Var}_{\text{instance}}/\mathrm{Var}_{\text{total}}$	Ratio	Cosine similarity
Baseline	16.5%	7.5%	2.2	0.50
Baseline + seg 1 forward	**48.8%**	**4.3%**	11.4	0.57
Baseline + attn 2 forward	**55.0%**	**5.5%**	10.0	0.55
Baseline + attn 2 forward & **64 feats**	**40.3%**	**5.8%**	**6.9**	**0.41**

Instance segmentation performance

Now set the cluster boundaries at the predicted semantic types



All:
ARI 0.38->0.53 near vertex
ARI 0.97->0.99 overall
>3% less UC

Shower:
ARI 0.06->0.40 near vertex
ARI 0.97->0.98 overall
>30% less UC

Clustering only done on the voxels of the same* ***predicted* *semantic type (lowE still excluded by truth)*

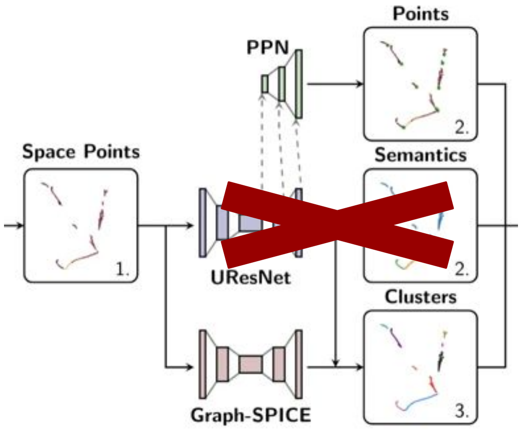
Conclusion and Outlook

- An iterative architecture of SPINE with the cross-talk between semantic and instance segmentation is shown, from which we achieved
  - **99% accuracy** in voxel segmentation **regardless of the distance to vertex**
  - ARI boosted by >0.3 in showers around the vertex, and overall **ARI=0.99 for all samples combined**
  - Much better decoder feature space that works for both the segmentation and clustering tasks

- Due to the nature of the two tasks, the decoder space is still dominated by the semantic segmentation task while the current GNN setup is not the optimal to extract the edges from decoder space and apply in 3D space
  - Ongoing work to enhance the instance task representation.
  - Experiment with radius (ball) graph or others to avoid the spatial distance asymmetry in KNN

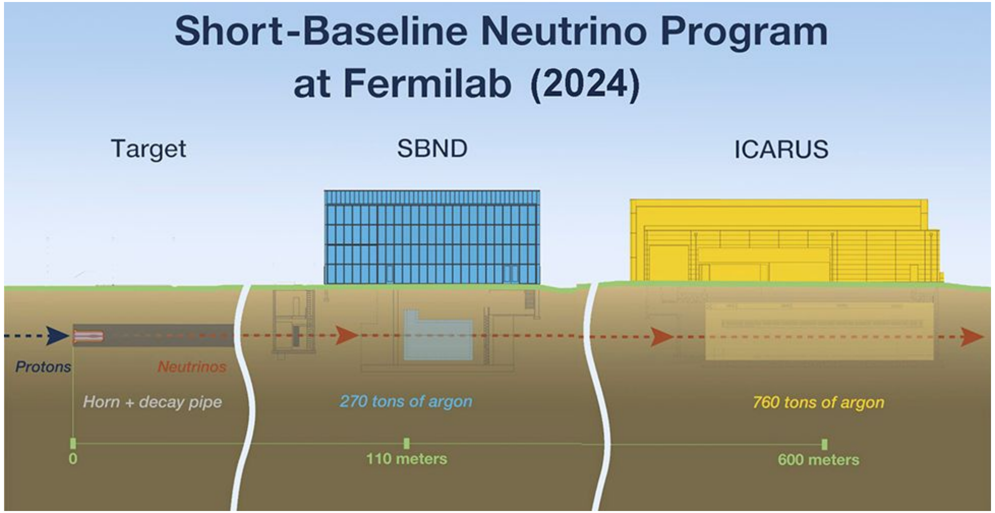
- Ongoing investigation of the performance over more iterations
  - Dive into the attention scores and understand the evolution

- Can simplify the current SPINE architecture
  - Remove the UResNet used just for 5-class semantic segmentation, and switch to the common encoder+decoder for PPN and Graph-SPICE
  - Apply to SBN event selection and analyses



Backup Slides

The Short Baseline Neutrino (SBN) Program



SBN [2021 ~]

- Situated in the BNB beamline, consists of the Short Baseline Near Detector (SBND) at 110 m and ICARUS detector at 600 m. Both detectors are LArTPC.
- Aims to address the LSND anomaly with the "near-far detectors" technique and simultaneous fit for appearance and disappearance channels.

Optical Readout in ICARUS

LAr has a large scintillation light yield at O(10k) per MeV

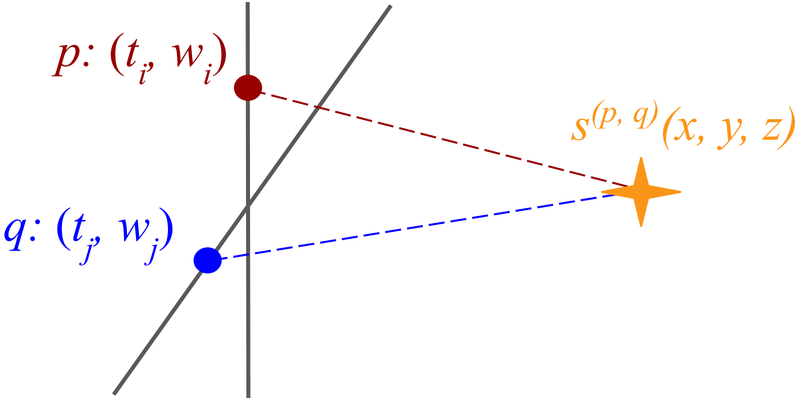
- LAr is transparent to its scintillation photons
- Fast component 4~8 ns
- Slow component 1.5~1.6 μs

360 PMTs behind the wire plane register optical information called “flash”

Require a **coincidence within 1.6 μs for the charge and optical readouts** to select neutrino interactions inside the detector.

From 2D wire hits to 3D spacepoints

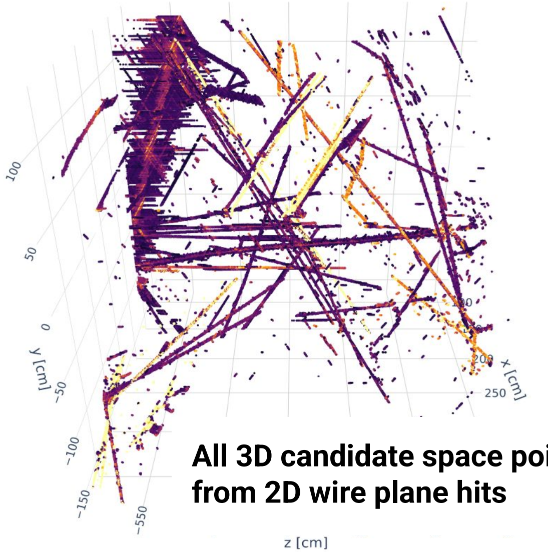
T. Usher



A wire hit on the wire plane p is defined by the hit time and wire ID

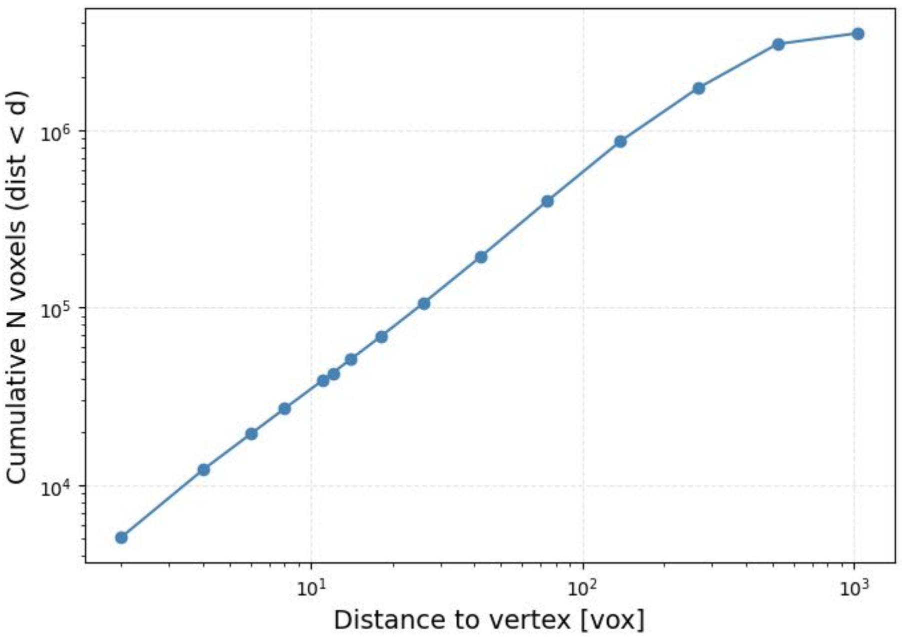
The 2 hits from different wire planes ($p\&q$) that have a hit time coincidence are used to reconstruct a 3D spacepoint.

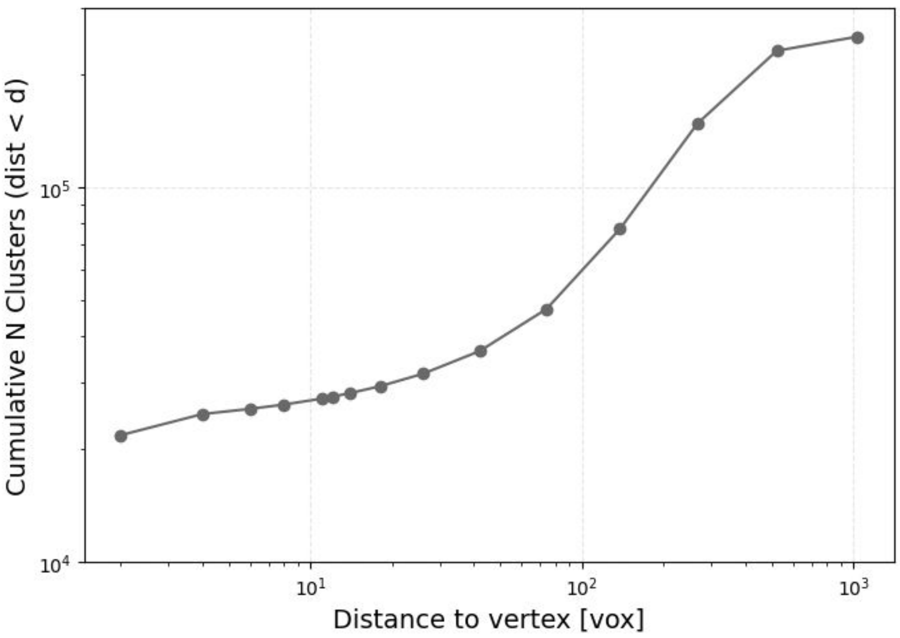
Since this is the very first step in event reconstruction, the algorithm (*Cluster3D*) is prioritized to maximize the efficiency while leave the spurious spacepoints "ghosts" to be removed in the downstream process.



All 3D candidate space points from 2D wire plane hits

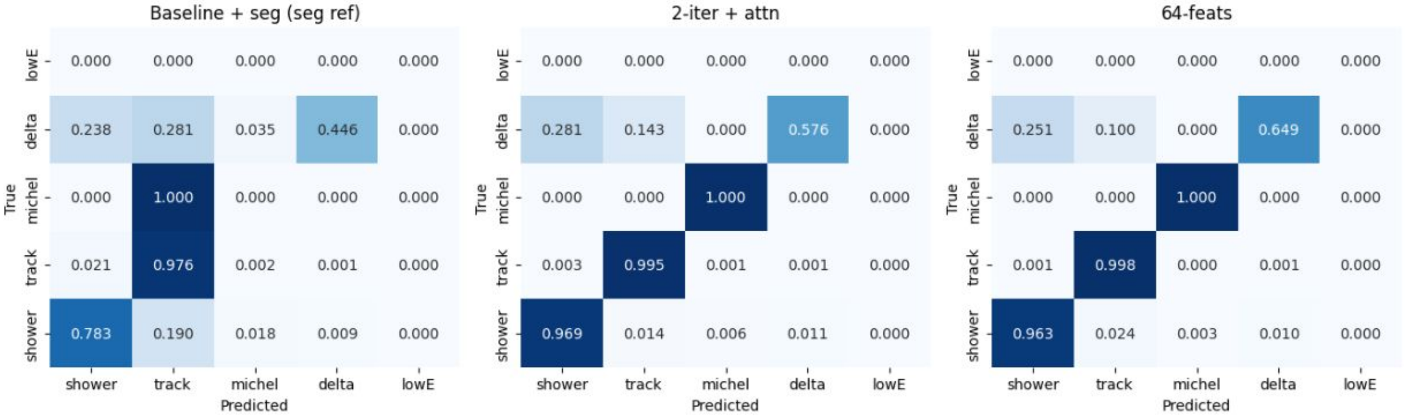
Validation stats



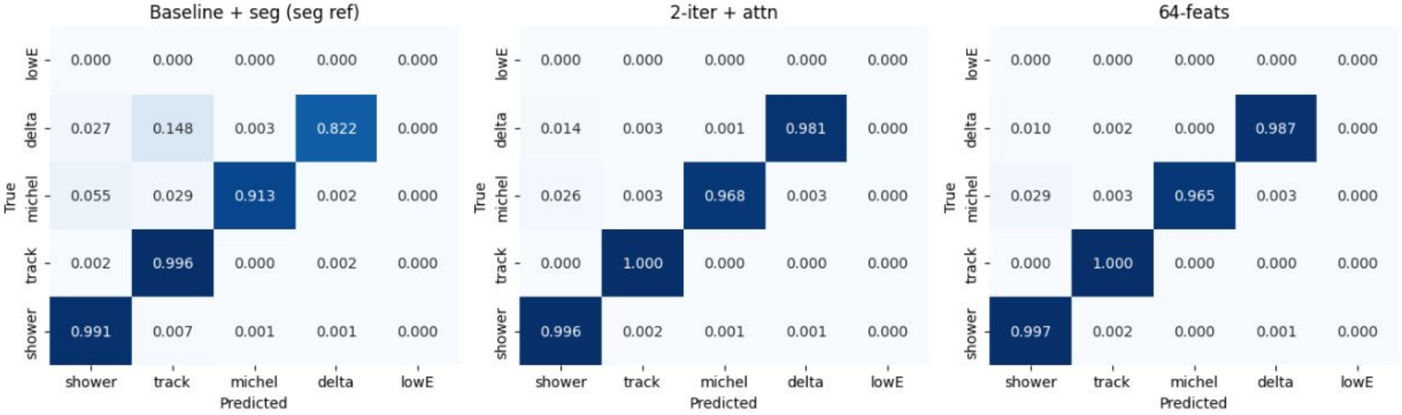


Semantic segmentation performance

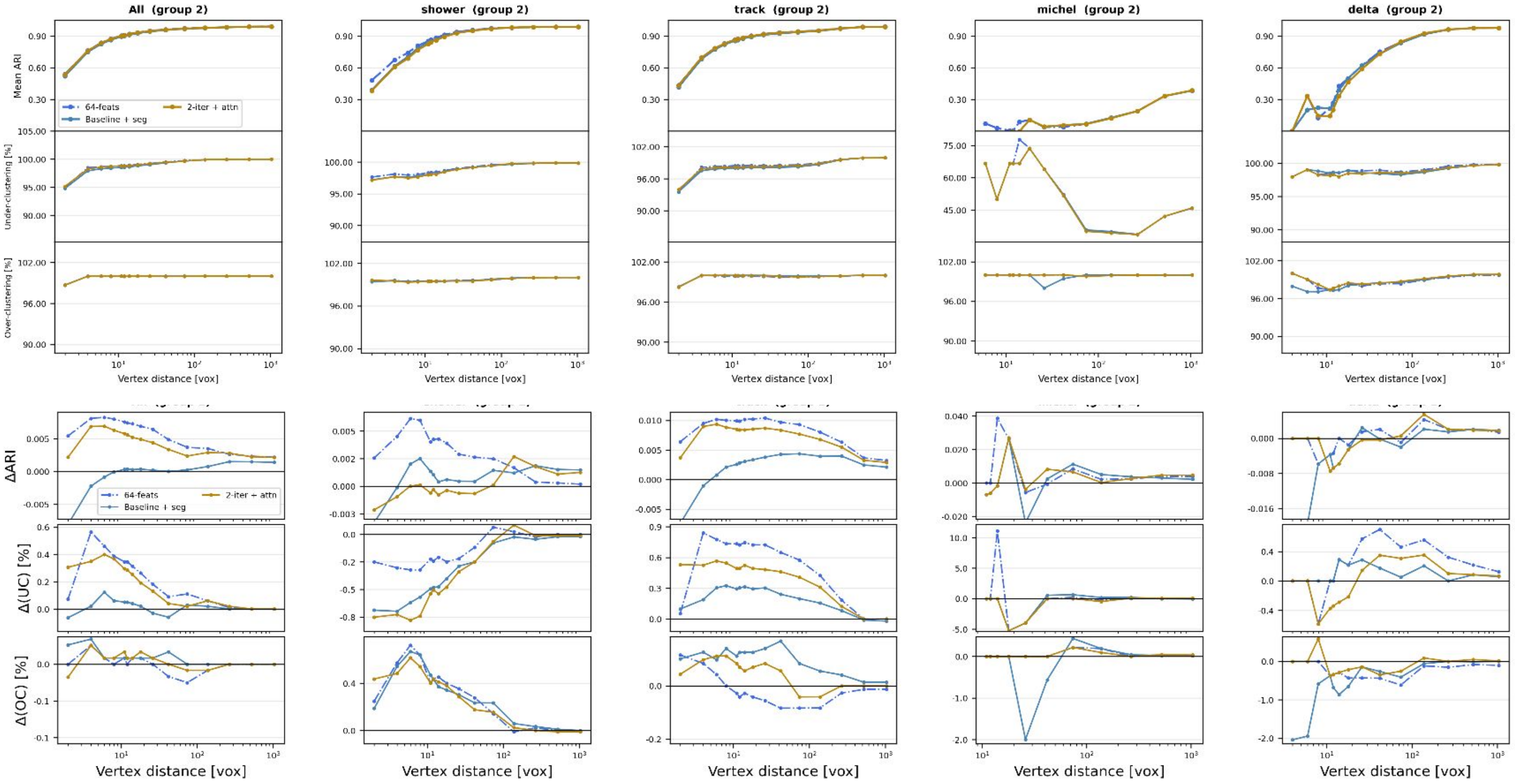
d<10 vox



d<1024 vox



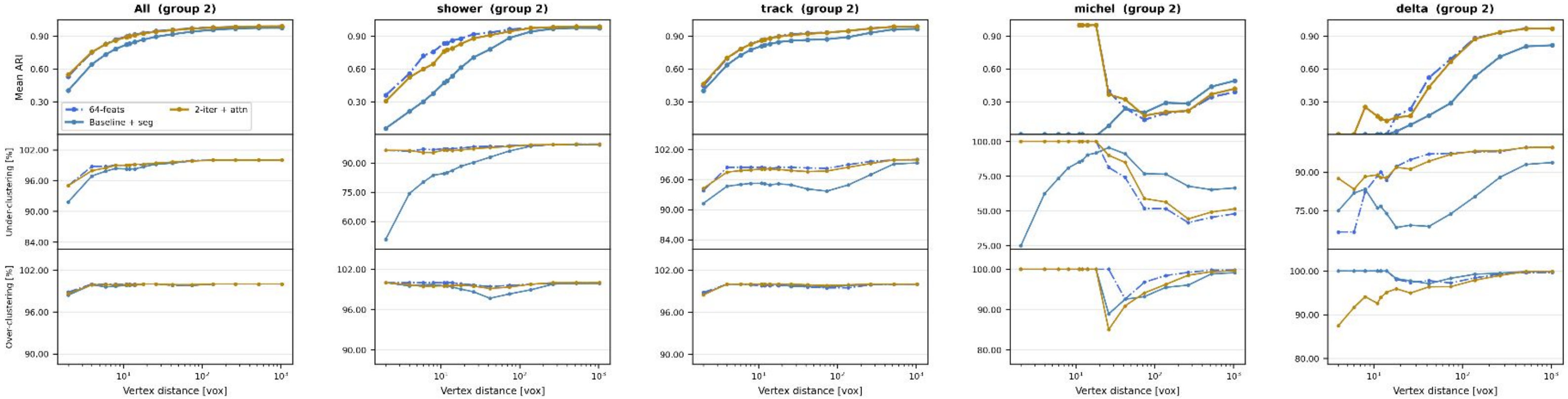
Instance segmentation performance



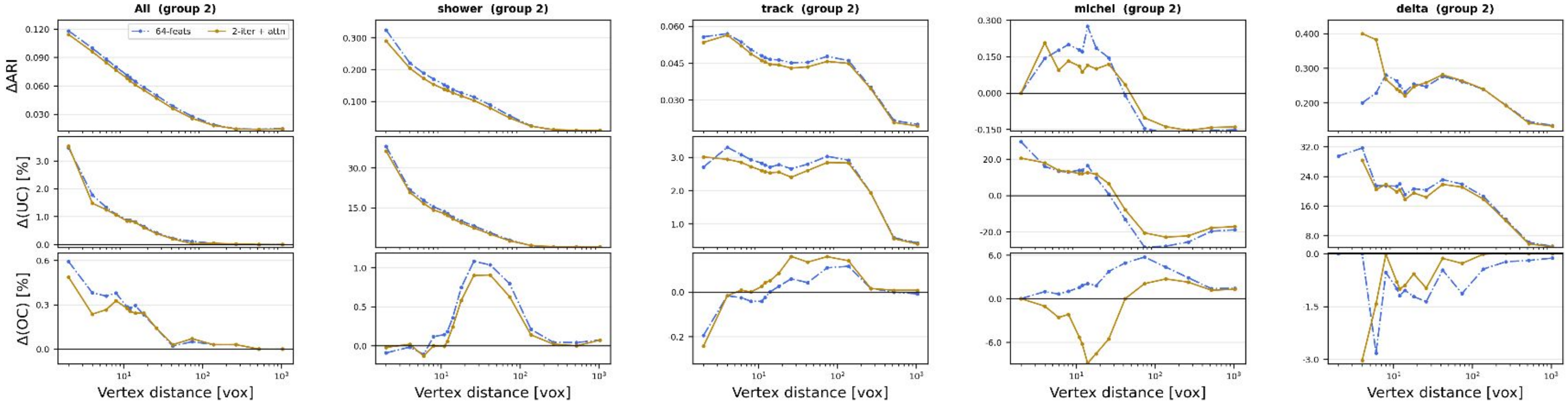
1 - OC/UC rate

*Clustering only done on the voxels of the same **truth** semantic type*

Instance segmentation performance



1 - OC/UC rate



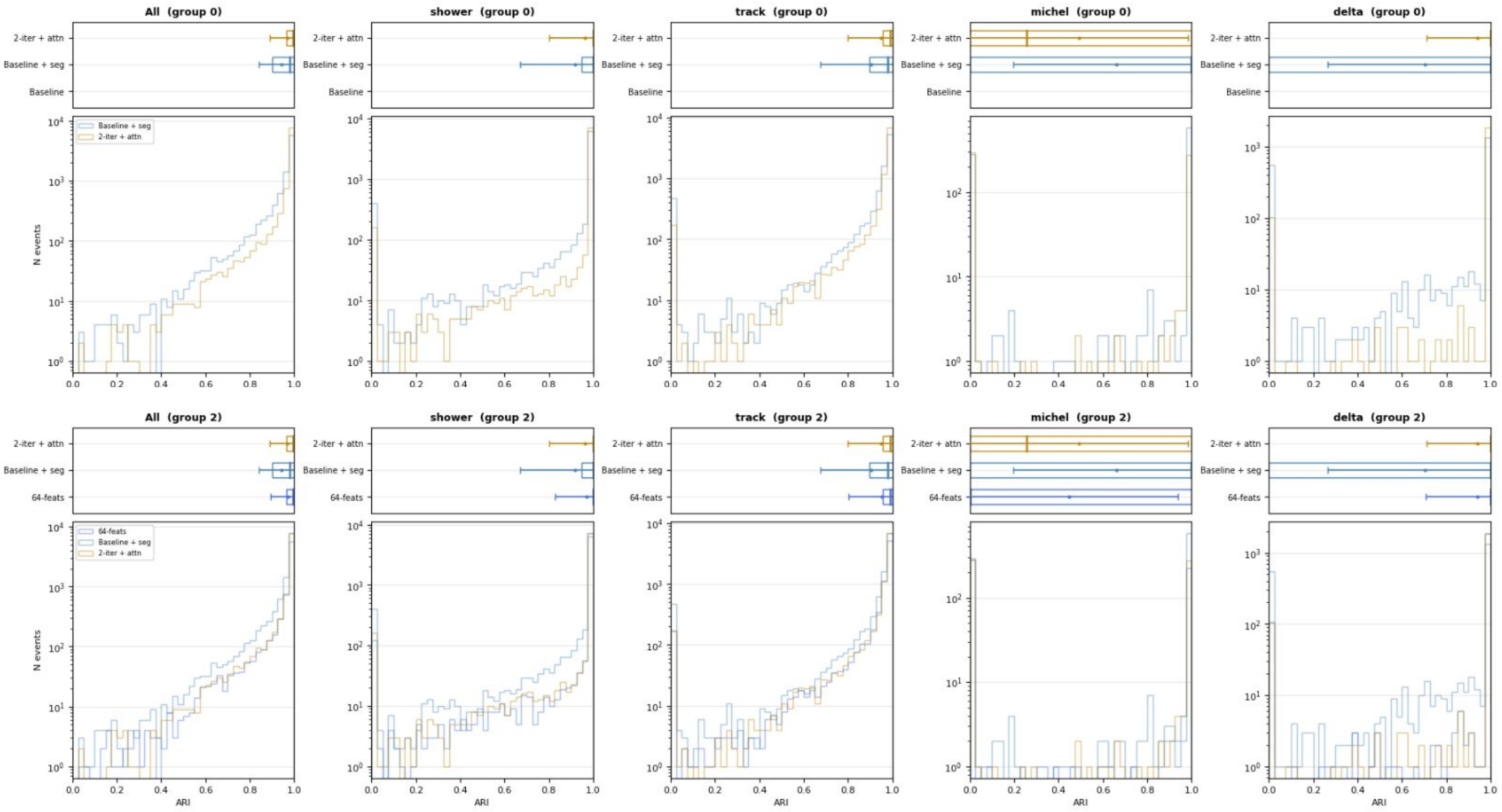
Clustering only done on the voxels of the same* ***predicted* *semantic type (lowE still excluded by truth)*

Instance segmentation performance

Mean ARI	All 4 classes	Shower	Track	Michel	Delta
1 forward	[0.38, 0.97]	[0.06, 0.97]	[0.39, 0.97]	[0.00, 0.48]	[0.01, 0.82]
2 forward + attn	**[0.52, 0.99]**	**[0.28, 0.98]**	**[0.43, 0.99]**	**[0.00, 0.40]**	**[0.00, 0.97]**
2 forward + attn & 64 feats	**[0.53, 0.99]**	**[0.40, 0.98]**	**[0.43, 0.99]**	**[0.00, 0.39]**	**[0.00, 0.97]**

Clustering only done on the voxels of the same* ***predicted* *semantic type (lowE still excluded by truth)*

Instance segmentation performance



*Clustering only done on the voxels of the same ***predicted*** semantic type (lowE still excluded by truth)*

Trade-off of semantic and instance segmentation

2 Nested Analysis of Variance (ANOVA)

2.1 Nested ANOVA Decomposition of Neural Feature Variance

1. Setup and Notation Let $\mathbf{x}_{k=1}^{N} \subset \mathbb{R}^{32}$ be the backbone feature vectors of N voxels collected across multiple events. Each voxel has two labels:

$s_k \in 0, 1, 2, 3$ — semantic type (shower, track, michel, delta) i_k — particle instance ID (unique per physical particle) Instances are nested within types: each physical particle belongs to exactly one semantic class, so the index set of instances is partitioned as $\mathcal{I} = \bigsqcup_s \mathcal{I}_s$. This is a balanced hierarchical design in classical ANOVA terminology (Montgomery, 2017, Ch. 13).

Define the mean vectors at each level:

$$\boldsymbol{\mu}_{\text{grand}} = \frac{1}{N} \sum_{k=1}^{N} \mathbf{x}_k$$

$$\boldsymbol{\mu}_s = \frac{1}{N_s} \sum_{k:s_k=s} \mathbf{x}_k, \quad N_s = |k : s_k = s|$$

$$\boldsymbol{\mu}_{s_i} = \frac{1}{N_{si}} \sum_{k:s_k=s,,i_k=i} \mathbf{x}_k, \quad N_{si} = |k : s_k = s, i_k = i|$$

2. Total Scatter and its Decomposition Following Mardia, Kent & Bibby (1979, Ch. 12), define the total scatter matrix:

$$\mathbf{T} = \sum_{k=1}^{N} (\mathbf{x}_k - \boldsymbol{\mu}_{\text{grand}})(\mathbf{x}_k - \boldsymbol{\mu}_{\text{grand}})^\top$$

The nested structure allows an exact algebraic partition $\mathbf{T} = \mathbf{B}_{\text{type}} + \mathbf{B}_{\text{inst}} + \mathbf{W}$ (Duda, Hart & Stork, 2001, §3.8.3):

$$\mathbf{B}_{\text{type}} = \sum_s N_s (\boldsymbol{\mu}_s - \boldsymbol{\mu}_{\text{grand}})(\boldsymbol{\mu}_s - \boldsymbol{\mu}_{\text{grand}})^\top$$

$$\mathbf{B}_{\text{inst}} = \sum_s \sum_{i \in \mathcal{I}_s} N_{si} (\boldsymbol{\mu}_{si} - \boldsymbol{\mu}_s)(\boldsymbol{\mu}_{si} - \boldsymbol{\mu}_s)^\top$$

$$\mathbf{W} = \sum_s \sum_{i \in \mathcal{I}\, s} \sum_{k:s_k=s,,i_k=i} (\mathbf{x}_k - \boldsymbol{\mu}_{si})(\mathbf{x}_k - \boldsymbol{\mu}_{si})^\top$$

The partition is exact because the three subspaces are mutually orthogonal — a direct consequence of the projector identity

$$\mathbf{x}_k - \boldsymbol{\mu}_{\text{grand}} = \underbrace{(\boldsymbol{\mu}_s - \boldsymbol{\mu}_{\text{grand}})}_{\perp\, \mathbf{B}_{\text{inst}}, \mathbf{W}} + \underbrace{(\boldsymbol{\mu}_{si} - \boldsymbol{\mu}_s)}_{\perp\, \mathbf{B}_{\text{type}}, \mathbf{W}} + \underbrace{(\mathbf{x}_k - \boldsymbol{\mu}_{si})}_{\perp\, \mathbf{B}_{\text{type}}, \mathbf{B}_{\text{inst}}}$$

Taking the trace (which is the total variance up to a factor $1/N$):

$$\underbrace{\text{tr}(\mathbf{T})/N}_{V_{\text{total}}} = \underbrace{\text{tr}(\mathbf{B}_{\text{type}})/N}_{V_{\text{semantic}}} + \underbrace{\text{tr}(\mathbf{B}_{\text{inst}})/N}_{V_{\text{instance}}} + \underbrace{\text{tr}(\mathbf{W})/N}_{V_{\text{noise}}}$$

Trade-off of semantic and instance segmentation

2.3 Linear Discriminant Subspaces

The scatter matrices define the Fisher linear discriminant subspaces (Duda, Hart & Stork, 2001, §3.8):

Semantic LDA subspace $\mathbf{U}_{\text{sem}} \in \mathbb{R}^{32\times(S-1)}$: the $S-1 = 3$ directions that maximize

$$J_{\text{sem}}(\mathbf{u}) = \frac{\mathbf{u}^\top \mathbf{B}_{\text{type}} \mathbf{u}}{\mathbf{u}^\top \mathbf{W} \mathbf{u}}$$

found by solving the generalized eigenvalue problem $\mathbf{B}_{\text{type}}\mathbf{u} = \lambda \mathbf{W}\mathbf{u}$.

2.5.2 Subspace overlap

The mean principal angle cosine between the semantic LDA subspace $\mathbf{U}_{\text{sem}}$ and the instance LDA subspace $\mathbf{U}_{\text{inst}}$:

$$\text{overlap} = \left(\mathbf{U}_{\text{sem}}^\top \mathbf{U}_{\text{inst}}\right) \in [0, 1]$$

1. = 0: orthogonal subspaces — semantic and instance tasks use **different** feature dimensions, no competition
2. = 1: identical subspaces — both tasks compete for the same dimensions (superposition)

If overlap increases with supervision → linear superposition is the mechanism behind the trade-off.

LDA finds the direction $\mathbf{u} \in \mathbb{R}^{32}$ that maximizes the Fisher criterion:

$$J(\mathbf{u}) = \frac{\mathbf{u}^\top \mathbf{S}_B \mathbf{u}}{\mathbf{u}^\top \mathbf{S}_W \mathbf{u}}$$

Take the derivative, set to zero:

$$\frac{\partial J}{\partial \mathbf{u}} = \frac{2\mathbf{S}_B\mathbf{u}(\mathbf{u}^\top \mathbf{S}_W \mathbf{u}) - 2\mathbf{S}_W\mathbf{u}(\mathbf{u}^\top \mathbf{S}_B \mathbf{u})}{(\mathbf{u}^\top \mathbf{S}_W \mathbf{u})^2} = \mathbf{0}$$

Defining $\lambda = J(\mathbf{u})$ and simplifying:

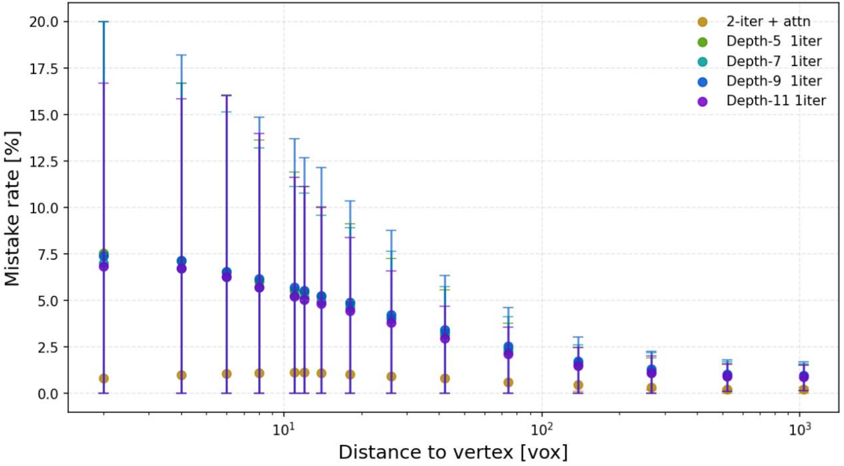
$$\mathbf{S}_B\mathbf{u} = \lambda \mathbf{S}_W \mathbf{u}$$

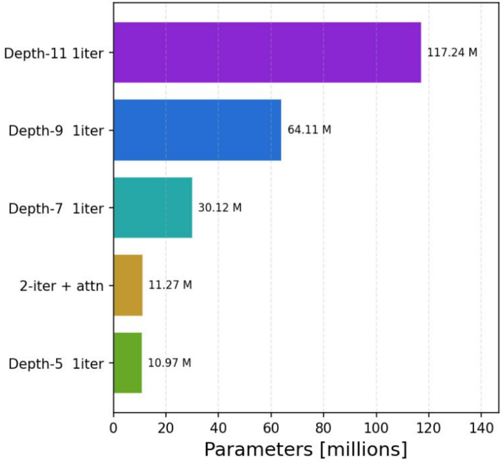
This is the generalized eigenvalue problem. Since $\mathbf{S}_W$ is positive definite (enforced by regularization), left-multiply both sides by $\mathbf{S}_W^{-1}$:

$$\underbrace{\mathbf{S}_W^{-1}\mathbf{S}_B}_{\mathbf{M}}\,\mathbf{u} = \lambda \mathbf{u}$$

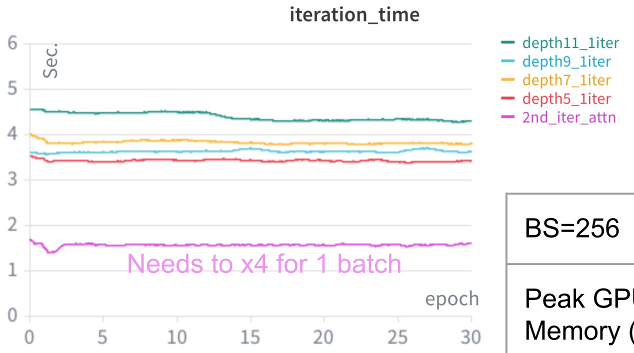
Now it is a standard eigenvalue problem. The eigenvectors of $\mathbf{M}$ are the LDA directions; the eigenvalue λ_i equals the Fisher ratio $J(\mathbf{u}_i)$ achieved in that direction.

Computational cost for the attention





Attention adds ~0.3M parameters to the default GraphSPICE, whereas deeper models cannot achieve similar segmentation performance.



Compromising computation time for GPU memory due to the attention, but not much:

BS=256	2-iter + attn	1-iter depth-5	1-iter depth-7	1-iter depth-9	1-iter depth-11
Peak GPU Memory (GB)	7.5 (16 events)	4.9	5.7	6.4	7.3